Simple Command Sequencer

Component Design Document

1 Description

The Command Sequencer component executes predefined sequences of commands. It receives high-level sequence commands and breaks them down into individual sub-commands that are sent to the command router. The sequencer handles command response tracking, timeouts, and failure modes.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *active*
- **Number of Connectors** - 8
- **Number of Invokee Connectors** - 3
- **Number of Invoker Connectors** - 5
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - 3
- **Number of Parameters** - *None*
- **Number of Events** - 17
- **Number of Faults** - *None*
- **Number of Data Products** - *None*
- **Number of Data Dependencies** - *None*
- **Number of Packets** - 1

3.2 Diagram

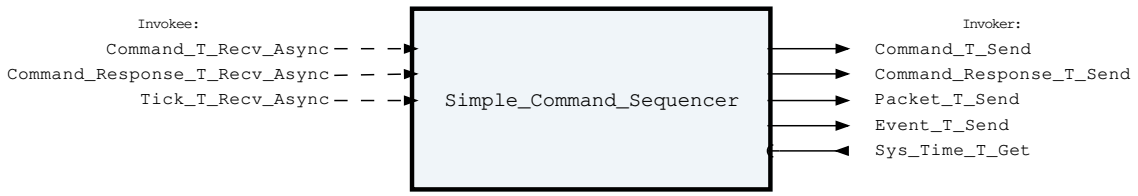

Figure 1: Simple Command Sequencer component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Simple Command Sequencer Invokee Connectors

Name	Kind	Type	Return_Type	Count
Command_T_Recv_Async	recv_async	Command.T	-	1
Command_Response_T_Recv_Async	recv_async	Command_Response.T	-	1
Tick_T_Recv_Async	recv_async	Tick.T	-	1

Connector Descriptions:

- **Command_T_Recv_Async** - Sequence commands are received on this connector
- **Command_Response_T_Recv_Async** - Responses to sub-commands are received here
- **Tick_T_Recv_Async** - Tick for managing timeouts and delays

3.3.2 Internal Queue

This component contains an internal first-in-first-out (FIFO) queue to handle asynchronous messages. This queue is sized at initialization as a configurable number of bytes. Determining the size of the component queue can be difficult. The following table lists the connectors that will put asynchronous messages onto the queue, and the maximum sizes of each of those messages on the queue. Note that each message put onto the queue also incurs an overhead on the queue of 5 additional bytes, which is included in the max message size below:

Table 2: Simple Command Sequencer Asynchronous Connectors

Name	Type	Max Size (bytes)
Command_T_Recv_Async	Command.T	265
Command_Response_T_Recv_Async	Command_Response.T	12
Tick_T_Recv_Async	Tick.T	17

If you are unsure how to size the queue of this component, it is recommended that you make the queue size a multiple of the largest size found above.

3.3.3 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 3: Simple Command Sequencer Invoker Connectors

Name	Kind	Type	Return_Type	Count
Command_T_Send	send	Command.T	-	1
Command_Response_T_Send	send	Command_Response.T	-	1
Packet_T_Send	send	Packet.T	-	1
Event_T_Send	send	Event.T	-	1
Sys_Time_T_Get	get	-	Sys_Time.T	1

Connector Descriptions:

- **Command_T_Send** - Sub-commands are sent out this connector
- **Command_Response_T_Send** - Sends the response to a Run_Sequence (or per-sequence wrapper) command back to the command router. The Send_After_Sequence_Completion path emits its deferred reply here on completion / abort / timeout / kill; the default Send_After_Sequence_Start path emits its reply here immediately at dispatch time.
- **Packet_T_Send** - The packet send connector, used for sending the periodic sequencer summary packet.
- **Event_T_Send** - Events are sent out of this connector
- **Sys_Time_T_Get** - The system time is retrieved via this connector.

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component achieves base class initialization using the `init_Base` subprogram. This subprogram requires the following parameters:

Table 4: Simple Command Sequencer Base Initialization Parameters

Name	Type
Queue_Size	Natural

Parameter Descriptions:

- **Queue_Size** - The number of bytes that can be stored in the component's internal queue.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 5: Simple Command Sequencer Set Id Bases Parameters

Name	Type
`Command_Id_Base`	`Command_Types.Command_Id_Base`
`Event_Id_Base`	`Event_Types.Event_Id_Base`
`Packet_Id_Base`	`Packet_Types.Packet_Id_Base`

Parameter Descriptions:

- **`Command_Id_Base`** - The value at which the component's command identifiers begin.
- **`Event_Id_Base`** - The value at which the component's event identifiers begin.
- **`Packet_Id_Base`** - The value at which the component's unresolved packet identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains no data dependencies.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. The component achieves implementation class initialization using the `init` subprogram. The `init` subprogram requires the following parameters:

Table 6: Simple Command Sequencer Implementation Initialization Parameters

Name	Type	Default Value
`Num_Concurrent_Sequences`	`Simple_Sequencer_ Types.Num_Concurrent_ Sequences_Type`	*None provided*
`Sequences`	`Simple_Sequencer_ Types.Sequences_ Access`	*None provided*

Parameter Descriptions:

- **`Num_Concurrent_Sequences`** - Denotes the Number of Sequences that can be running at the same time. Any Run_Sequence commands that are sent that would increase the number of concurrent sequences beyond this number will be rejected. The type bounds this to the number of frames whose summaries fit in one summary packet.
- **`Sequences`** - Access to statically defined sequence list.

3.6 Commands

Static commands for the Simple Command Sequencer component. Per-sequence wrapper commands (one per declared sequence) are synthesised at assembly load time by gen/models/simple_sequencer_commands.py.

Table 7: Simple Command Sequencer Commands

Local ID	Command Name	Argument Type
0	Run_Sequence	Run_Sequence_Arg.T
1	Kill_All_Sequences	-
2	Set_Summary_Packet_Period	Packed_U16.T

Command Descriptions:

- **Run_Sequence** - Run a command sequence by ID. Per-sequence wrapper commands are the operator-friendly form; this is the underlying backbone they all dispatch through.
- **Kill_All_Sequences** - Halt every running sequence and return all frames to their initial state. Does not affect frames that were not running.
- **Set_Summary_Packet_Period** - Set the period of the sequencer summary packet, in ticks. A period of zero disables the packet.

3.7 Parameters

The Simple Command Sequencer component has no parameters.

3.8 Events

Events for the Simple Command Sequencer component

Table 8: Simple Command Sequencer Events

Local ID	Event Name	Parameter Type
0	Sequence_Started	Sequence_Event_Info.T
1	Sequence_Completed	Sequence_Event_Info.T
2	Sequence_Aborted	Sequence_Step_Event_Info.T
3	Sequence_Timeout	Sequence_Step_Event_Info.T
4	Sequence_Out_Of_Range_Sleep	Sequence_Sleep_Event_Info.T
5	Sequence_Out_Of_Range_Timeout	Sequence_Timeout_Event_ Info.T
6	Command_Failure	Sequence_Step_Command_ Event_Info.T
7	Invalid_Sequence_Id	Packed_U32.T
8	Extra_Sequence_Id	-
9	No_Frame_Available	-
10	Dropped_Command	Command_Header.T
11	Dropped_Command_Response	Command_Response.T
12	Dropped_Tick	Tick.T
13	Invalid_Command_Received	Invalid_Command_Info.T
14	Unexpected_Command_Response	Command_Response.T
15	Killed_All_Sequences	-
16	Invalid_Dynamic_Command_Argument	Sequence_Step_Command_ Event_Info.T

Event Descriptions:

- **Sequence_Started** - A sequence has begun execution on a frame
- **Sequence_Completed** - A sequence has completed all steps successfully
- **Sequence_Aborted** - A sequence was aborted due to a command failure
- **Sequence_Timeout** - A sequence timed out while waiting for a command response

- **Sequence_Out_Of_Range_Sleep** - A sequence has entered sleep state with either overflow or underflow.
- **Sequence_Out_Of_Range_Timeout** - A sequence timeout duration is out of range either underflow or overflow.
- **Command_Failure** - A command sent by a sequence received a failure response
- **Invalid_Sequence_Id** - A Run_Sequence command was received with an out of range sequence ID
- **Extra_Sequence_Id** - Too many sequence IDs were passed to the Simple Command Sequencer
- **No_Frame_Available** - A Run_Sequence command was received but all frames are in use
- **Dropped_Command** - A command was dropped due to a full queue
- **Dropped_Command_Response** - A command response was dropped due to a full queue
- **Dropped_Tick** - A tick was dropped due to a full queue
- **Invalid_Command_Received** - A command was received with invalid parameters.
- **Unexpected_Command_Response** - A command response was received with a source ID that does not belong to any active sequence frame.
- **Killed_All_Sequences** - A Kill_All_Sequences command was executed and all running sequences were halted.
- **Invalid_Dynamic_Command_Argument** - A Command with a Dynamic Argument cannot be executed as the Argument is Invalid

3.9 Data Products

The Simple Command Sequencer component has no data products.

3.10 Data Dependencies

The Simple Command Sequencer component has no data dependencies.

3.11 Packets

Packets for the Simple Command Sequencer component.

Table 9: Simple Command Sequencer Packets

Local ID	Packet Name	Type
0x0000 (0)	Summary_Packet	*Undefined*

Packet Descriptions:

- **Summary_Packet** - Periodic summary of all sequence frames. Contains one Sequence_Frame_Summary record per frame (Num_Concurrent_Sequences total, in frame order), reporting which sequence is running on each frame, its current step, its execution state, and whether an operator is still waiting on a deferred command response. Its type is determined dynamically based on the Num_Concurrent_Sequences declared for the component instance in the assembly. Emitted every Summary_Packet_Period ticks; a period of zero (the default) disables emission. The period is set with the Set_Summary_Packet_Period command.

3.12 Faults

The Simple Command Sequencer component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Simple_ Command_ Sequencer_ Tests* Test Suite

This is a unit test suite for the Simple Command Sequencer component

Test Descriptions:

- **`Test_Start_Sequence`** - *No description provided.*
- **`Test_Dynamic_Sequence`** - *No description provided.*
- **`Test_Invalid_Sequence_Id`** - *No description provided.*
- **`Test_No_Frame_Available`** - *No description provided.*
- **`Test_Command_Failure_Aborts_Sequence`** - *No description provided.*
- **`Test_Command_Failure_Continues_Sequence`** - *No description provided.*
- **`Test_Concurrent_Sequences`** - *No description provided.*
- **`Test_Spurious_Command_Response`** - *No description provided.*
- **`Test_Tick_Does_Not_Wake_Waiting_For_Resp`** - *No description provided.*
- **`Test_Dropped_Command`** - *No description provided.*
- **`Test_Dropped_Command_Response`** - *No description provided.*
- **`Test_Dropped_Tick`** - *No description provided.*
- **`Test_Invalid_Command`** - *No description provided.*
- **`Test_No_Wait_Sequence`** - *No description provided.*
- **`Test_Frame_Reuse_After_Completion`** - *No description provided.*
- **`Test_Out_Of_Range_Sleep`** - *No description provided.*
- **`Test_Timeout`** - *No description provided.*
- **`Test_Out_Of_Range_Timeout`** - *No description provided.*
- **`Test_Extra_Sequence_Id`** - *No description provided.*
- **`Test_Kill_All_Sequences`** - *No description provided.*
- **`Test_Set_Summary_Packet_Period`** - *No description provided.*
- **`Test_Synthesized_Sequence_Command`** - *No description provided.*
- **`Test_Deferred_Response_On_Completion`** - *No description provided.*
- **`Test_Deferred_Response_On_Abort`** - *No description provided.*
- **`Test_Deferred_Response_On_Timeout`** - *No description provided.*
- **`Test_Deferred_Response_On_Kill_All`** - *No description provided.*
- **`Test_Concurrent_Deferred_Responses`** - *No description provided.*
- **`Test_Sub_Sequence_Call`** - *No description provided.*
- **`Test_Summary_Packet`** - *No description provided.*

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Command.T:

Generic command packet for holding arbitrary commands

Table 10: Command Packed Record : 2080 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Command_ Header.T	-	40	0	39	-
Arg_Buffer	Command_ Types. Command_Arg_ Buffer_Type	-	2040	40	2079	Header.Arg_ Buffer_Length

Field Descriptions:

- **Header** - The command header
- **Arg_Buffer** - A buffer that contains the command arguments

Command_Header.T:

Generic command header for holding arbitrary commands

Table 11: Command_Header Packed Record : 40 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_Types. Command_Source_Id	0 to 65535	16	0	15
Id	Command_Types. Command_Id	0 to 65535	16	16	31
Arg_Buffer_Length	Command_Types. Command_Arg_Buffer_ Length_Type	0 to 255	8	32	39

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Id** - The command identifier
- **Arg_Buffer_Length** - The number of bytes used in the command argument buffer

Command_Response.T:

Record for holding command response data.

Table 12: Command_Response Packed Record : 56 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_ Types.Command_ Source_Id	0 to 65535	16	0	15
Registration_ Id	Command_ Types.Command_ Registration_ Id	0 to 65535	16	16	31
Command_Id	Command_Types. Command_Id	0 to 65535	16	32	47
Status	Command_Enums. Command_ Response_ Status.E	0 => Success 1 => Failure 2 => Id_Error 3 => Validation_Error 4 => Length_Error 5 => Dropped 6 => Register 7 => Register_Source	8	48	55

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Registration_Id** - The registration ID. An ID assigned to each registered component at initialization.
- **Command_Id** - The command ID for the command response.
- **Status** - The command execution status.

Event.T:

Generic event packet for holding arbitrary events

Table 13: Event Packed Record : 344 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Event_Header.T	-	88	0	87	-
Param_Buffer	Event_Types. Parameter_ Buffer_Type	-	256	88	343	Header.Param_ Buffer_Length

Field Descriptions:

- **Header** - The event header
- **Param_Buffer** - A buffer that contains the event parameters

Event_Header.T:

Generic event packet for holding arbitrary events

Table 14: Event_Header Packed Record : 88 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Id	Event_Types.Event_Id	0 to 65535	16	64	79
Param_Buffer_Length	Event_Types.Parameter_Buffer_Length_Type	0 to 32	8	80	87

Field Descriptions:

- **Time** - The timestamp for the event.
- **Id** - The event identifier
- **Param_Buffer_Length** - The number of bytes used in the param buffer

Invalid_Command_Info.T:

Record for holding information about an invalid command

Table 15: Invalid_Command_Info Packed Record : 112 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Command_Types.Command_Id	0 to 65535	16	0	15
Errant_Field_Number	Interfaces.Unsigned_32	0 to 4294967295	32	16	47
Errant_Field	Basic_Types.Poly_Type	-	64	48	111

Field Descriptions:

- **Id** - The command Id received.
- **Errant_Field_Number** - The field that was invalid. 1 is the first field, 0 means unknown field, 2**32 means that the length field of the command was invalid.
- **Errant_Field** - A polymorphic type containing the bad field data, or length when Errant_Field_Number is 2**32.

Packed_U16.T:

Single component record for holding packed unsigned 16-bit value.

Table 16: Packed_U16 Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Interfaces.Unsigned_16	0 to 65535	16	0	15

Field Descriptions:

- **Value** - The 16-bit unsigned integer.

Packed_U32.T:

Single component record for holding packed unsigned 32-bit value.

Table 17: Packed_U32 Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Interfaces. Unsigned_32	0 to 4294967295	32	0	31

Field Descriptions:
- **Value** - The 32-bit unsigned integer.

Packet.T:

Generic packet for holding arbitrary data

Table 18: Packet Packed Record : 10080 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Packet_ Header.T	-	112	0	111	-
Buffer	Packet_ Types.Packet_ Buffer_Type	-	9968	112	10079	Header. Buffer_Length

Field Descriptions:
- **Header** - The packet header
- **Buffer** - A buffer that contains the packet data

Packet_Header.T:

Generic packet header for holding arbitrary data

Table 19: Packet_Header Packed Record : 112 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Id	Packet_Types. Packet_Id	0 to 65535	16	64	79
Sequence_Count	Packet_Types. Sequence_Count_Mod_ Type	0 to 16383	16	80	95
Buffer_Length	Packet_Types. Packet_Buffer_ Length_Type	0 to 1246	16	96	111

Field Descriptions:
- **Time** - The timestamp for the packet item.
- **Id** - The packet identifier
- **Sequence_Count** - Packet Sequence Count
- **Buffer_Length** - The number of bytes used in the packet buffer

Run_Sequence_Arg.T:

Argument Type for Run Sequence Command

Table 20: Run_Sequence_Arg Packed Record : 2040 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Sequence_Id	Interfaces.Unsigned_16	0 to 65535	16	0	15	-
Response_Behavior	Sequence_Enums.Sequence_Response_Behavior.E	0 => Send_After_Sequence_Start 1 => Send_After_Sequence_Completion	8	16	23	-
Arg_Length	Simple_Sequencer_Types.Run_Sequence_Arg_Buffer_Length_Type	0 to 250	16	24	39	-
Buffer_Arg	Simple_Sequencer_Types.Run_Sequence_Buffer_Type	-	2000	40	2039	Arg_Length

Field Descriptions:

- **Sequence_Id** - Id of a Command Sequence Registered with the Simple Command Sequencer
- **Response_Behavior** - When the sequencer should reply to this Run_Sequence command - immediately on start or after the sequence completes.
- **Arg_Length** - Length of the buffer argument in bytes
- **Buffer_Arg** - Passthrough Argument for Commands in Sequence

Sequence_Event_Info.T:

Basic sequence and frame identification for events

Table 21: Sequence_Event_Info Packed Record : 64 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces.Unsigned_32	0 to 4294967295	32	0	31
Frame_Id	Interfaces.Unsigned_32	0 to 4294967295	32	32	63

Field Descriptions:

- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on

Sequence_Sleep_Event_Info.T:

Sequence sleep event information

Table 22: Sequence_Sleep_Event_Info Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Frame_Id	Interfaces. Unsigned_32	0 to 4294967295	32	32	63
Milliseconds	Interfaces. Unsigned_32	0 to 4294967295	32	64	95

Field Descriptions:
- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on
- **Milliseconds** - The number of milliseconds to sleep

Sequence_Step_Command_Event_Info.T:

Sequence, frame, step, and command identification for events

Table 23: Sequence_Step_Command_Event_Info Packed Record : 112 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Frame_Id	Interfaces. Unsigned_32	0 to 4294967295	32	32	63
Step	Interfaces. Unsigned_32	0 to 4294967295	32	64	95
Command_Id	Command_Types. Command_Id	0 to 65535	16	96	111

Field Descriptions:
- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on
- **Step** - The step index
- **Command_Id** - The ID of the command

Sequence_Step_Event_Info.T:

Sequence, frame, and step identification for events

Table 24: Sequence_Step_Event_Info Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Frame_Id	Interfaces. Unsigned_32	0 to 4294967295	32	32	63
Step	Interfaces. Unsigned_32	0 to 4294967295	32	64	95

Field Descriptions:

- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on
- **Step** - The step index

Sequence_Timeout_Event_Info.T:

Sequence sleep event information

Table 25: Sequence_Timeout_Event_Info Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Frame_Id	Interfaces. Unsigned_32	0 to 4294967295	32	32	63
Milliseconds	Interfaces. Unsigned_32	0 to 4294967295	32	64	95

Field Descriptions:

- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on
- **Milliseconds** - The number of milliseconds to sleep

Sys_Time.T:

A record which holds a time stamp using GPS format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 26: Sys_Time Packed Record : 64 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_32	0 to 4294967295	32	32	63

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^32) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 27: Tick Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Count	Interfaces. Unsigned_32	0 to 4294967295	32	64	95

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Command_Enums.Command_Response_Status.E:

This status enumeration provides information on the success/failure of a command through the command response connector.

Table 28: Command_Response_Status Literals:

Name	Value	Description
Success	0	Command was passed to the handler and successfully executed.
Failure	1	Command was passed to the handler not successfully executed.
Id_Error	2	Command id was not valid.
Validation_Error	3	Command parameters were not successfully validated.
Length_Error	4	Command length was not correct.
Dropped	5	Command overflowed a component queue and was dropped.
Register	6	This status is used to register a command with the command routing system.
Register_Source	7	This status is used to register command sender's source id with the command router for command response forwarding.

Sequence_Enums.Sequence_Response_Behavior.E:

Table 29: Sequence_Response_Behavior Literals:

Name	Value	Description
Send_After_Sequence_Start	0	
Send_After_Sequence_Completion	1	